Monitoring Intelligence Analyst Challenge

3.2 - Get your hands dirty

For the Checkouts analysis, SQLite was used:

For the Checkouts analysis, SQLite was used, considering variations of:

- Today
- Yesterday
- Same_day_last_week

With references to month and year averages for context.

- Minimum Relative Variation: 50% above or below the averages of the last week and month
- Absolute Variation: 10 transactions (to avoid false positives in low volumes)

Note: Some column names were renamed for better understanding during code development, with assistance from artificial intelligences for structuring and validation.
A graph was also made using Google Sheets to show the values of the two databases throughout the day

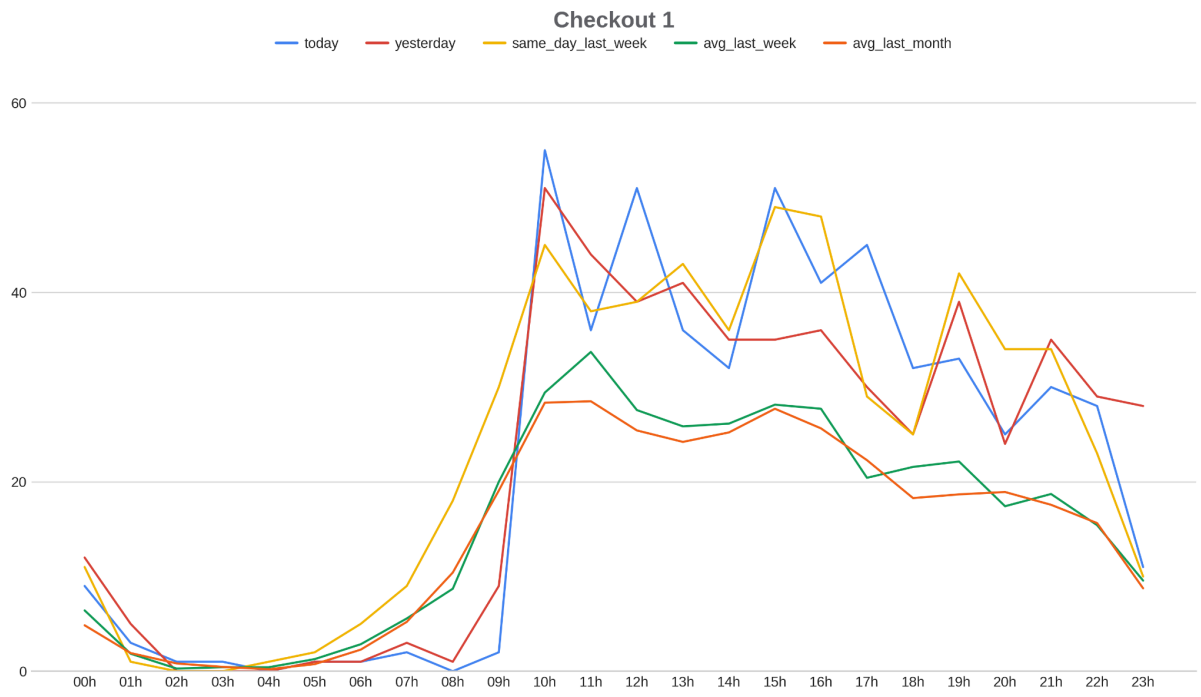
```
WITH base AS (

SELECT
    hour,
    today,
    yesterday,
    same_day_last_week,
    avg_last_week,
    avg_last_month,

    -- TODAY
    today - avg_last_week AS diff_today_week,
    today - avg_last_month AS diff_today_month,
    today / NULLIF(avg_last_week,0) AS ratio_today_week,
    today / NULLIF(avg_last_month,0) AS ratio_today_month,

    -- YESTERDAY
    yesterday - avg_last_week AS diff_yesterday_week,
    yesterday - avg_last_month AS diff_yesterday_month,
    yesterday / NULLIF(avg_last_week,0) AS ratio_yesterday_week,
    yesterday / NULLIF(avg_last_month,0) AS ratio_yesterday_month,
```

```
    -- SAME DAY LAST WEEK
    same_day_last_week - avg_last_week AS diff_lastweek_week,
    same_day_last_week - avg_last_month AS diff_lastweek_month,
    same_day_last_week / NULLIF(avg_last_week,0) AS ratio_lastweek_week,
    same_day_last_week / NULLIF(avg_last_month,0) AS ratio_lastweek_month

FROM checkout_1

)

SELECT
    hour,
    today,
    yesterday,
    same_day_last_week,
    avg_last_week,
    avg_last_month,

    -- TODAY classification
    CASE
        WHEN (ratio_today_week > 1.5 OR ratio_today_month > 1.5)
        AND (diff_today_week > 10 OR diff_today_month > 10)
        THEN 'SPIKE'

        WHEN (ratio_today_week < 0.5 OR ratio_today_month < 0.5)
        AND (diff_today_week < -10 OR diff_today_month < -10)
        THEN 'DROP'

        ELSE 'NORMAL'
    END AS anomaly_today,

    -- YESTERDAY classification
    CASE
        WHEN (ratio_yesterday_week > 1.5 OR ratio_yesterday_month > 1.5)
        AND (diff_yesterday_week > 10 OR diff_yesterday_month > 10)
        THEN 'SPIKE'

        WHEN (ratio_yesterday_week < 0.5 OR ratio_yesterday_month < 0.5)
        AND (diff_yesterday_week < -10 OR diff_yesterday_month < -10)
        THEN 'DROP'

        ELSE 'NORMAL'
```

```
  END AS anomaly_yesterday,

  -- SAME DAY LAST WEEK classification
  CASE
    WHEN (ratio_lastweek_week > 1.5 OR ratio_lastweek_month > 1.5)
    AND (diff_lastweek_week > 10 OR diff_lastweek_month > 10)
    THEN 'SPIKE'

    WHEN (ratio_lastweek_week < 0.5 OR ratio_lastweek_month < 0.5)
    AND (diff_lastweek_week < -10 OR diff_lastweek_month < -10)
    THEN 'DROP'

    ELSE 'NORMAL'
  END AS anomaly_same_day_last_week

FROM base
ORDER BY hour;
```

Checkout 1:

hour	today	yesterday	same_day_last_week	avg_last_week	avg_last_month	anomaly_today	anomaly_yesterday	anomaly_same_day_last_week
00h	9	12	11	6.42	4.85	NORMAL	NORMAL	NORMAL
01h	3	5	1	1.85	1.92	NORMAL	NORMAL	NORMAL
02h	1	0	0	0.28	0.82	NORMAL	NORMAL	NORMAL
03h	1	0	0	0.42	0.46	NORMAL	NORMAL	NORMAL
04h	0	0	1	0.42	0.21	NORMAL	NORMAL	NORMAL
05h	1	1	2	1.28	0.75	NORMAL	NORMAL	NORMAL
06h	1	1	5	2.85	2.28	NORMAL	NORMAL	NORMAL
07h	2	3	9	5.57	5.21	NORMAL	NORMAL	NORMAL
08h	0	1	18	8.71	10.42	DROP	NORMAL	NORMAL
09h	2	9	30	20	19.07	DROP	DROP	SPIKE
10h	55	51	45	29.42	28.35	SPIKE	SPIKE	SPIKE
11h	36	44	38	33.71	28.5	NORMAL	SPIKE	NORMAL
12h	51	39	39	27.57	25.42	SPIKE	SPIKE	SPIKE
13h	36	41	43	25.85	24.21	NORMAL	SPIKE	SPIKE
14h	32	35	36	26.14	25.21	NORMAL	NORMAL	NORMAL
15h	51	35	49	28.14	27.71	SPIKE	NORMAL	SPIKE
16h	41	36	48	27.71	25.64	SPIKE	NORMAL	SPIKE
17h	45	30	29	20.42	22.28	SPIKE	NORMAL	NORMAL
18h	32	25	25	21.57	18.28	SPIKE	NORMAL	NORMAL
19h	33	39	42	22.14	18.67	SPIKE	SPIKE	SPIKE
20h	25	24	34	17.42	18.92	NORMAL	NORMAL	SPIKE
21h	30	35	34	18.71	17.57	SPIKE	SPIKE	SPIKE
22h	28	29	23	15.42	15.64	SPIKE	SPIKE	NORMAL
23h	11	28	10	9.57	8.75	NORMAL	SPIKE	NORMAL

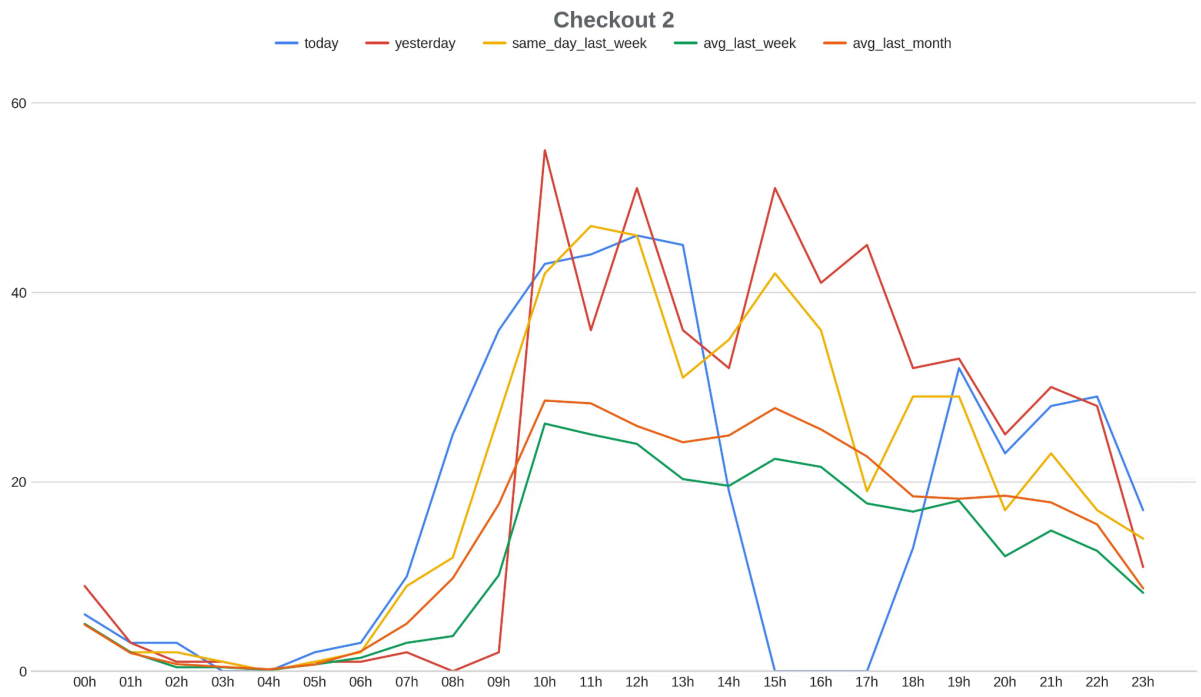


It can be observed that today and yesterday show a drop in transactions at 8 a.m., followed by a peak at 10 a.m.

Also for today, there was a peak between 3 p.m. and 5 p.m. and between 7 p.m. and 10 p.m., with the latter also occurring yesterday and on the same day last week.

Checkout 2:

hour	today	yesterday	same_day_last_week	avg_last_week	avg_last_month	anomaly_today	anomaly_yesterday	anomaly_same_day_last_week
00h	6	9	5	5	4.92	NORMAL	NORMAL	NORMAL
01h	3	3	2	2	1.92	NORMAL	NORMAL	NORMAL
02h	3	1	2	0.42	0.75	NORMAL	NORMAL	NORMAL
03h	0	1	1	0.42	0.46	NORMAL	NORMAL	NORMAL
04h	0	0	0	0.14	0.21	NORMAL	NORMAL	NORMAL
05h	2	1	1	0.71	0.71	NORMAL	NORMAL	NORMAL
06h	3	1	2	1.42	2.1	NORMAL	NORMAL	NORMAL
07h	10	2	9	3	5.03	NORMAL	NORMAL	NORMAL
08h	25	0	12	3.71	9.82	SPIKE	NORMAL	NORMAL
09h	36	2	27	10.14	17.64	SPIKE	DROP	SPIKE
10h	43	55	42	26.14	28.57	SPIKE	SPIKE	SPIKE
11h	44	36	47	25	28.28	SPIKE	NORMAL	SPIKE
12h	46	51	46	24	25.89	SPIKE	SPIKE	SPIKE
13h	45	36	31	20.28	24.17	SPIKE	SPIKE	SPIKE
14h	19	32	35	19.57	24.89	NORMAL	SPIKE	SPIKE
15h	0	51	42	22.427	27.78	DROP	SPIKE	SPIKE
16h	0	41	36	21.57	25.53	DROP	SPIKE	SPIKE
17h	0	45	19	17.71	22.67	DROP	SPIKE	NORMAL
18h	13	32	29	16.85	18.46	NORMAL	SPIKE	SPIKE
19h	32	33	29	18	18.21	SPIKE	SPIKE	SPIKE
20h	23	25	17	12.14	18.53	SPIKE	SPIKE	NORMAL
21h	28	30	23	14.85	17.82	SPIKE	SPIKE	NORMAL
22h	29	28	17	12.71	15.5	SPIKE	SPIKE	NORMAL
23h	17	11	14	8.28	8.75	NORMAL	NORMAL	NORMAL



There were peaks between 10 a.m. and 2 p.m. for today, yesterday, and the same day last week, with yesterday maintaining the elevated behavior until the end of the day and last week normalizing after 7 p.m.

It shows irregularity between 3 p.m. and 5 p.m. that requires investigation, with peaks yesterday and in the previous week, and an interesting point was the lack of transactions for today.

3.2 - Solve the problem

A historical analysis of the available data was carried out to understand transaction behavior and define alert rules for each type of occurrence.

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Upload CSV file (used in Google Colab)
from google.colab import files
uploaded = files.upload()


# Load dataset
transactions = pd.read_csv("transactions.csv")

# Sort transactions by timestamp for easier time-series analysis
transaction_ordenados = transactions.sort_values(by='timestamp')
transaction_ordenados.head()


# Convert timestamp column to datetime format
transactions["timestamp"] = pd.to_datetime(transactions["timestamp"])


# ------------------------------------------------------------------------------
# DATA PREPARATION
# ------------------------------------------------------------------------------

# Transform the dataset so that each transaction status becomes a column
# Each row will represent the number of transactions per minute per status

per_minute = (
    transactions
    .pivot_table(
        index="timestamp",   # time index
        columns="status",    # transaction status becomes columns
        values="count",      # number of transactions
```

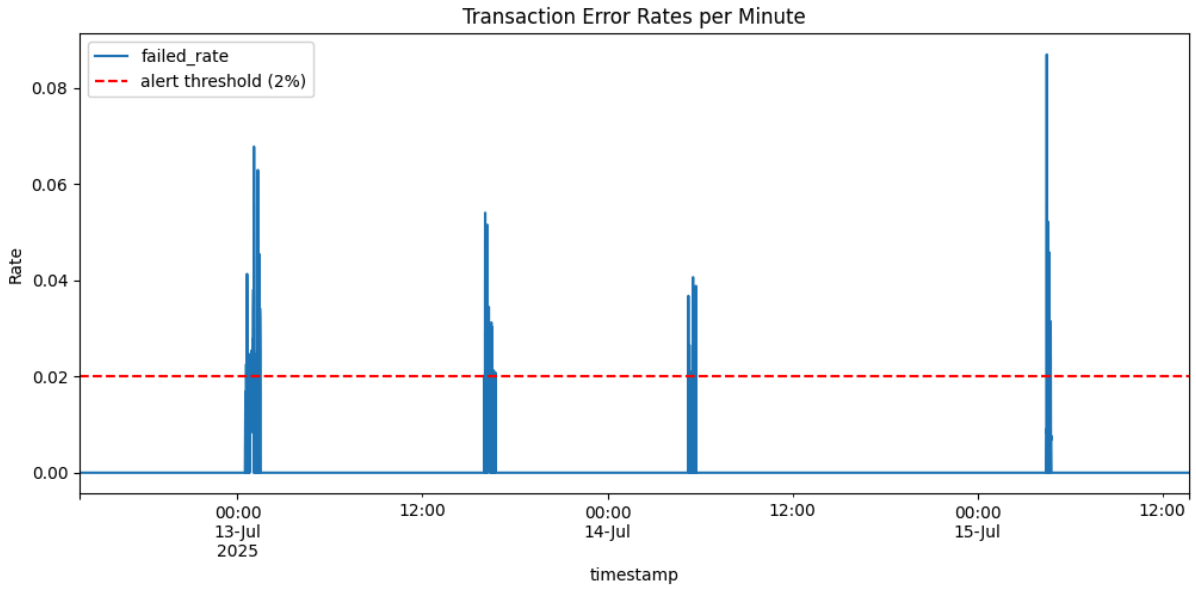
```
        aggfunc="sum"
    )
    .fillna(0)            # replace missing values with zero
)

per_minute.head()


# ---------------------------------------------------------------------------
# FEATURE ENGINEERING
# ---------------------------------------------------------------------------

# Calculate the total number of transactions per minute
per_minute["total"] = per_minute.sum(axis=1)


# Calculate error ratios relative to the total transaction volume

# Failed transactions ratio
per_minute["failed_rate"] = per_minute["failed"] / per_minute["total"]

# Denied transactions ratio
per_minute["denied_rate"] = per_minute["denied"] / per_minute["total"]

# Reversed transactions ratio
per_minute["reversed_rate"] = per_minute["reversed"] / per_minute["total"]

# Backend reversed transactions ratio
per_minute["backend_reversed_rate"] = per_minute["backend_reversed"] / per_minute["total"]


# ---------------------------------------------------------------------------
# ANOMALY DETECTION (RULE-BASED)
# ---------------------------------------------------------------------------

# The anomaly detection rules were defined after visually inspecting
# the error rate graphs generated below.
#
# The thresholds represent abnormal behavior relative to normal transaction patterns.
#
# Example:
# If the failed transaction rate exceeds 2% of total transactions in a minute,
```

```
# the system raises an alert.

# ---------------------------------------------------------------------------
# VISUAL ANALYSIS
# ---------------------------------------------------------------------------

# The following graphs help visualize transaction behavior over time.
# The dashed red line represents the anomaly threshold defined above.

# FAILED TRANSACTION RATE

per_minute[["failed_rate"]].plot(figsize=(12,5))

plt.axhline(
    y=0.02,
    color="red",
    linestyle="--",
    label="alert threshold (2%)"
)

plt.title("Failed Transaction Rate Monitoring")
plt.ylabel("Rate")
plt.legend()
plt.show()

# DENIED TRANSACTION RATE

per_minute[["denied_rate"]].plot(figsize=(12,5))

plt.axhline(
    y=0.10,
    color="red",
    linestyle="--",
    label="alert threshold (10%)"
)

plt.title("Denied Transaction Rate Monitoring")
plt.ylabel("Rate")
plt.legend()
plt.show()

# REVERSED TRANSACTION RATE
```

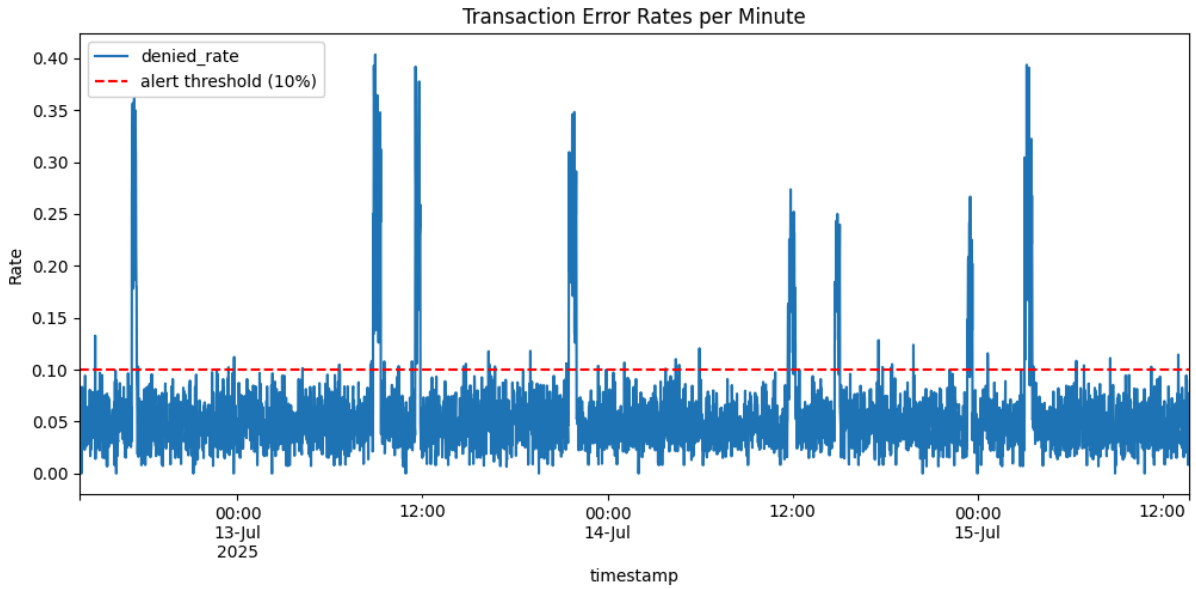
```
per_minute[["reversed_rate"]].plot(figsize=(12,5))

plt.axhline(
    y=0.03,
    color="red",
    linestyle="--",
    label="alert threshold (3%)"
)

plt.title("Reversed Transaction Rate Monitoring")
plt.ylabel("Rate")
plt.legend()
plt.show()

# BACKEND REVERSED RATE

per_minute[["backend_reversed_rate"]].plot(figsize=(12,5))

plt.axhline(
    y=0.01,
    color="red",
    linestyle="--",
    label="alert threshold (1%)"
)

plt.title("Backend Reversed Transaction Rate Monitoring")
plt.ylabel("Rate")
plt.legend()
plt.show()
```

Below are the generated charts and the visual line that defined the alert criteria rules based on the available historical data

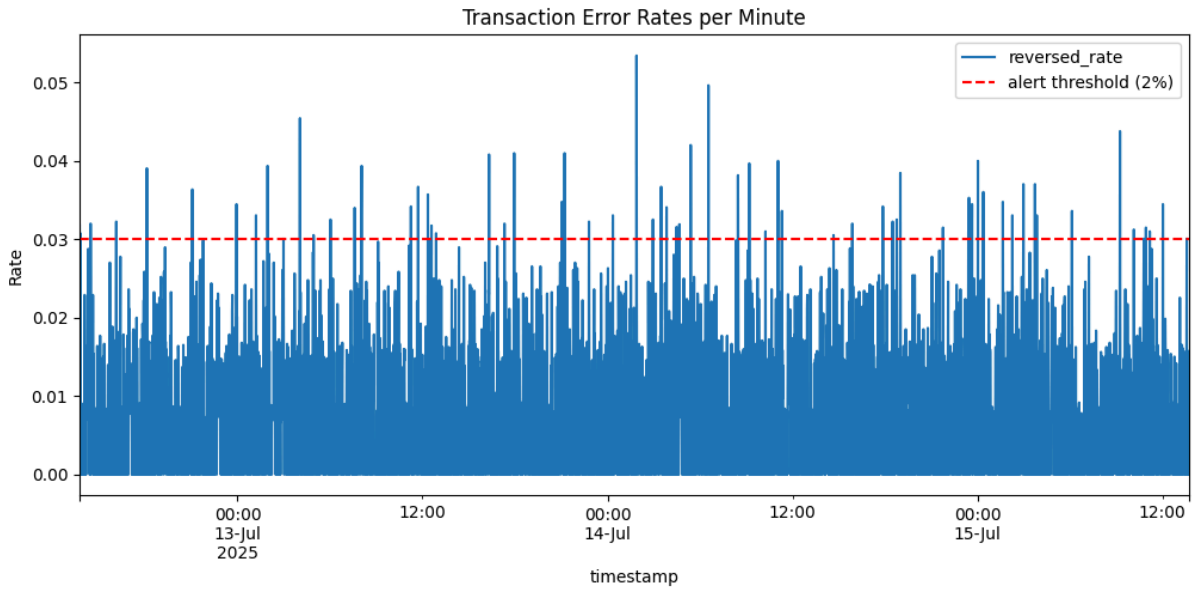
Failed Rate:



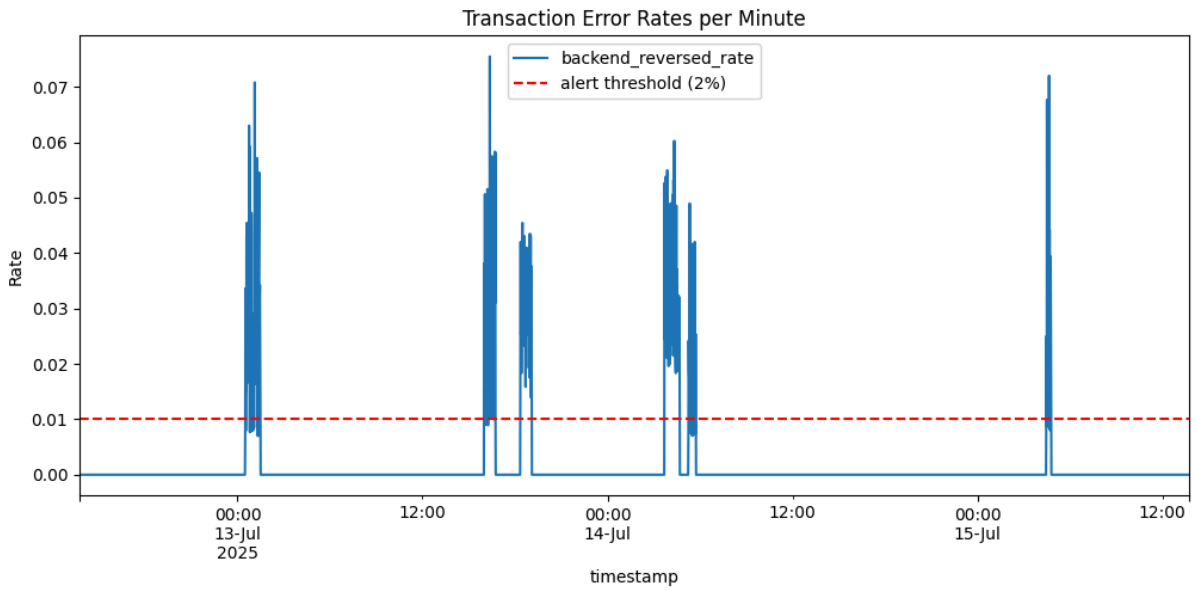
Denied rate:



Reversed Rate:



Backend Reversed Rate:



Defined Alert Thresholds:

Métrica	Threshold	Justification
Failed Rate	2%	Maximum acceptable failed transactions
Denied Rate	10%	Maximum acceptable denied transactions
Reversed Rate	3%	Maximum acceptable reversed transactions
Backend Reversed Rate	1%	Maximum acceptable backend reversals

The proposed monitoring system would be:

transactions
↓
data preparation (pivot_table)
↓
feature engineering (rates)
↓
rule-based anomaly detection
↓
create incidents table
↓
automatic alert reporting

1️⃣ Endpoint

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/monitor_transactions")
```

2️⃣ Data preparation and chart

```
per_minute = (
    transactions
    .pivot_table(
        index="timestamp",
        columns="status",
```

```
        values="count",
        aggfunc="sum"
    )
    .fillna(0)
)

per_minute[["failed_rate"]].plot(figsize=(12,5))

plt.axhline(
    y=0.02,
    color="red",
    linestyle="--",
    label="alert threshold (2%)"
)
plt.title("Failed Transaction Rate Monitoring")
plt.ylabel("Rate")
plt.legend()
plt.show()
```

3 Rule-based anomaly detection:

```
def monitor_transactions(data: dict):

    total = data["total"]

    failed_rate = data["failed"] / total
    denied_rate = data["denied"] / total
    reversed_rate = data["reversed"] / total
    backend_reversed_rate = data["backend_reversed"] / total

    if failed_rate > 0.02:
        return {"alert": True, "type": "ALERT_FAILED"}

    if denied_rate > 0.10:
        return {"alert": True, "type": "ALERT_DENIED"}

    if reversed_rate > 0.03:
        return {"alert": True, "type": "ALERT_REVERSED"}

    if backend_reversed_rate > 0.01:
        return {"alert": True, "type": "ALERT_BACKEND_REVERSED"}

    return {"alert": False}
```

Aplication

```
per_minute["alert"] = per_minute.apply(detect_anomaly, axis=1)
```

4 Automatic Alert System:

```
incidents = per_minute[per_minute["alert"] != "NORMAL"]
```

```
def report_incidents(df):

    if df.empty:
        print("No incidents detected")

    else:

        for timestamp, row in df.iterrows():

            alert_type = row["alert"]

            print(
                f"ALERT DETECTED | time: {timestamp} | type: {alert_type}"
            )


report_incidents(incidents)
```